

Comparing Neural Networks and Polynomial Regression for Function Approximation

Carter Glover

4/27

Goal

This project explores how model design choices affect function approximation performance, focusing on neural networks and polynomial regression across different function types.

Research Questions

When do neural networks outperform polynomial regression?

How does neural network architecture(depth, width) affect performance?

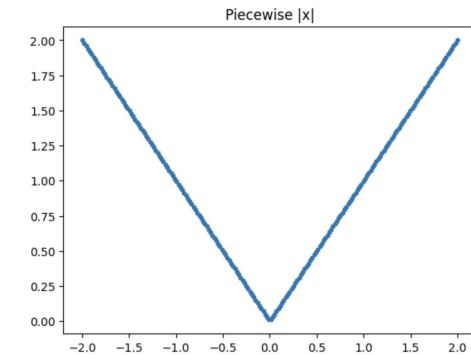
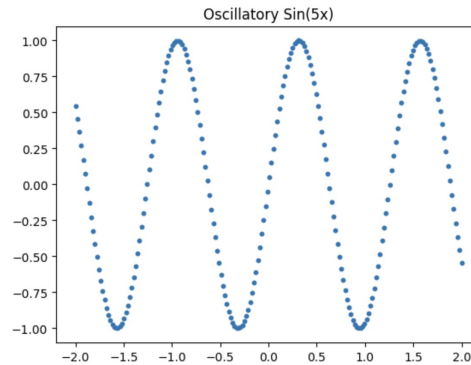
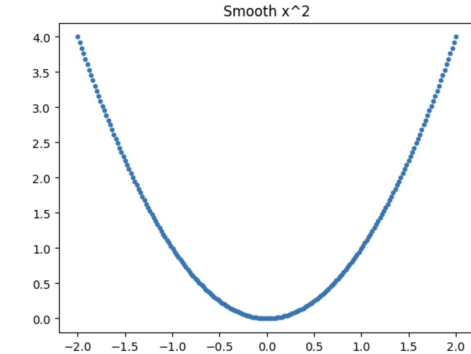
How do different function types (smooth, oscillatory, piecewise) influence model accuracy?

Functions

We test three types of functions:

- Smooth: x^2
- Oscillatory: $\sin(5x)$
- Piecewise: $|x|$

These represent increasing levels of complexity.



Methods/Experiment Design

Polynomial Regression

- Degree is varied to control complexity

Neural Networks

- Architecture (depth/width) is varied, fixed activation
- Activation functions (ReLU, Tanh, Sigmoid) are tested, fixed architecture
- Performance measured using Mean Squared Error (MSE)

Theoretical Background

Function Approximation:

- Learn a mapping $f(x) \rightarrow y$ from data
- Goal: approximate unknown functions using samples

Polynomial Regression:

- Models relationships using powers of input x
- Produces a single global function across the input range
- Behavior is controlled by polynomial degree

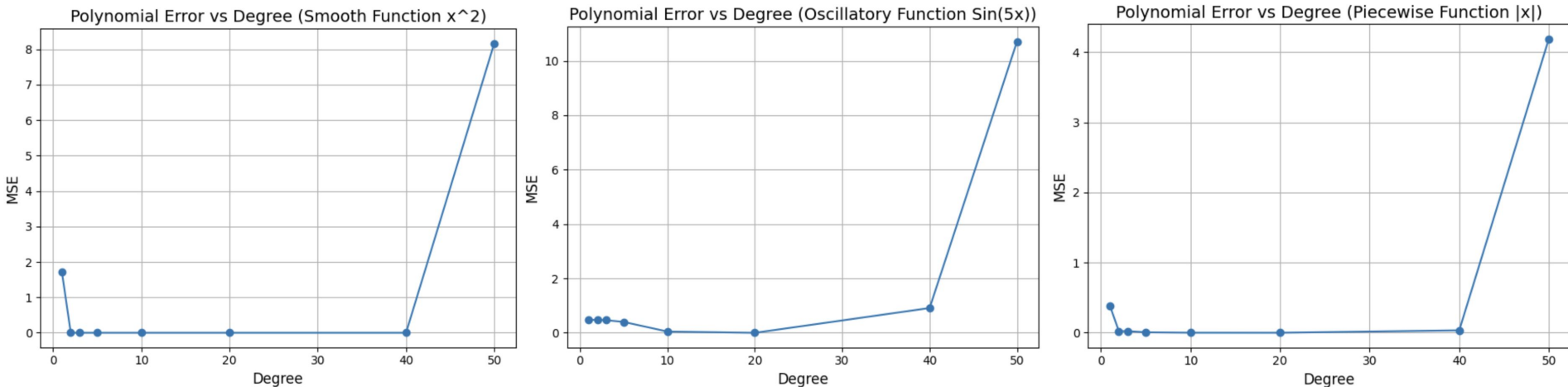
Neural Networks:

- Composed of layers of linear transformations and activation functions
- Each layer transforms inputs into new feature representations
- Structure is defined by number of layers and neurons

Activation Functions:

- Apply a nonlinear transformation to neuron outputs
- Enable the network to model nonlinear relationships
- Applied after each layer to transform features

Polynomial Regression Results

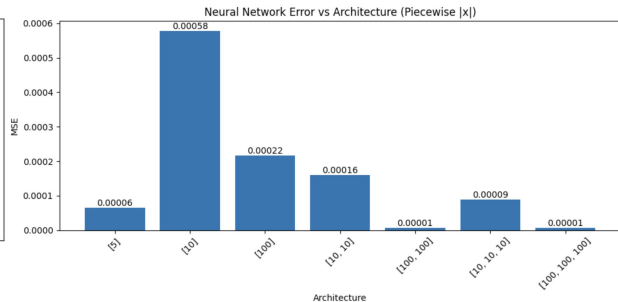
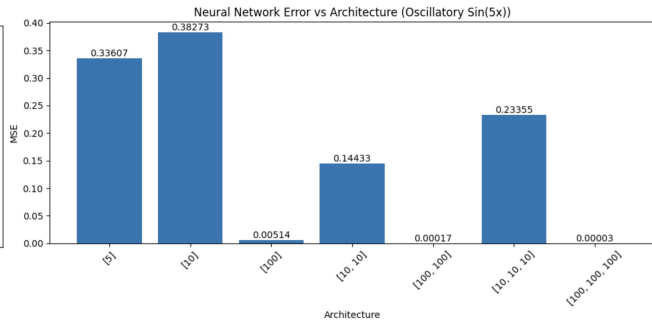
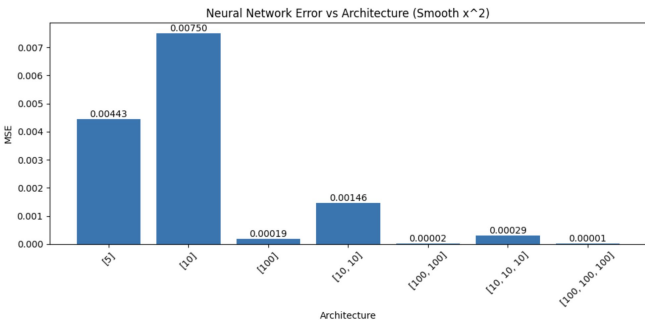


Key Observations:

degrees = [1, 2, 3, 5, 10, 20, 40, 50]

- Low-degree polynomials consistently underfit all functions due to limited flexibility
- Smooth functions are well approximated with relatively low-degree models
- Higher-degree polynomials are needed for oscillatory and piecewise functions but can introduce instability
- Polynomial regression performance becomes less reliable as function complexity increases

Neural Network Architecture Results



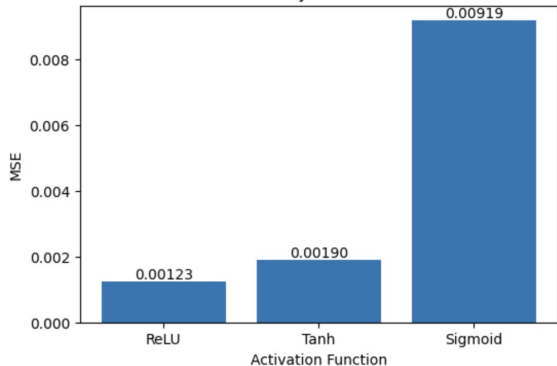
Activation = ReLU

Key Observations:

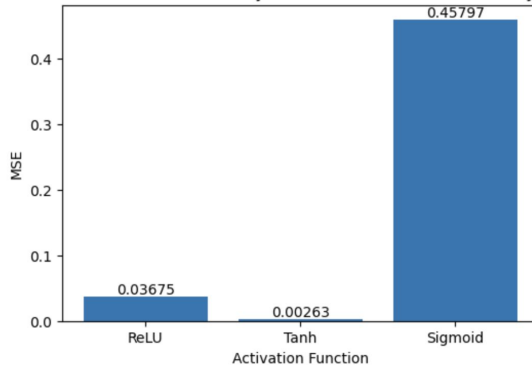
- Simple neural networks lack sufficient capacity to fully capture complex functions
- Increasing network size improves performance up to a point, after which gains become marginal
- Very large architectures do not guarantee better performance and may show diminishing returns
- Neural network performance depends on matching model capacity to function complexity
- Neural network results can vary slightly across training runs due to random initialization and optimization dynamics, even with fixed seeds

Activation Function Results

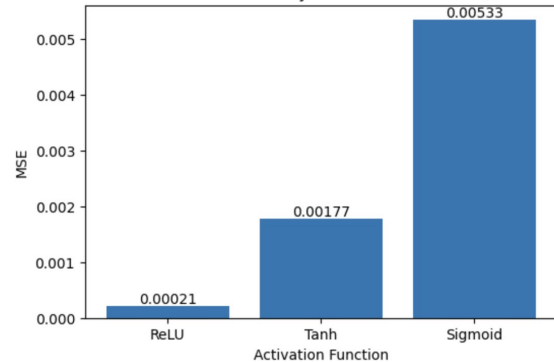
Neural Network Performance by Activation Function (Smooth x^2)



Neural Network Performance by Activation Function (Oscillatory $\sin(5x)$)



Neural Network Performance by Activation Function (Piecewise $|x|$)

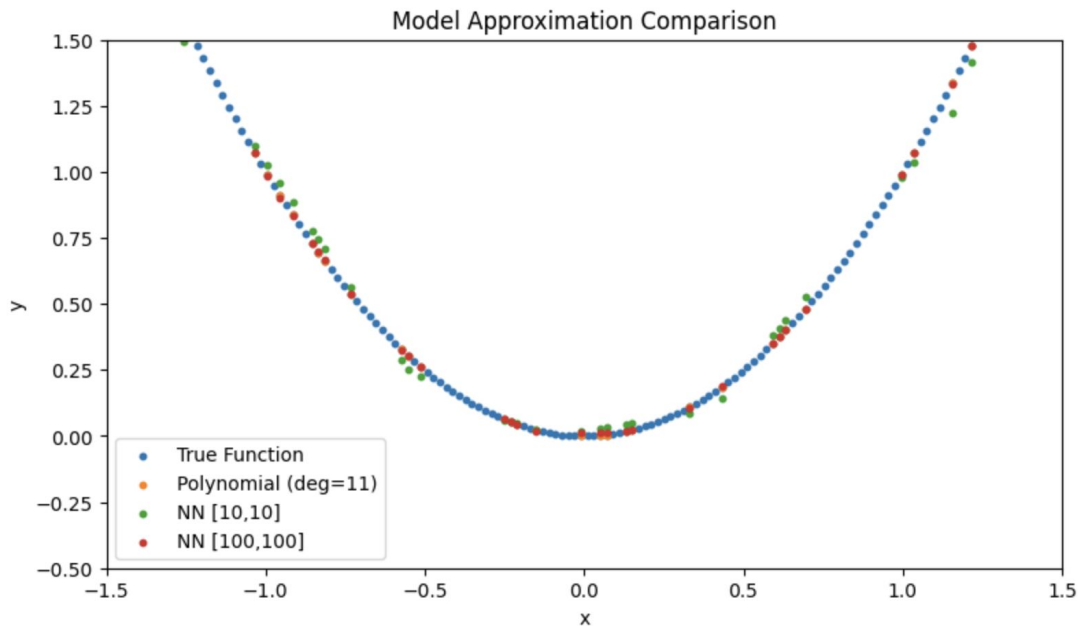


Key Observations:

Architecture = [10, 10]

- Activation functions influence model performance, but generally less than architecture
- ReLU performs consistently well across smooth and piecewise functions due to stable gradient behavior
- Tanh tends to perform better on oscillatory functions because it can represent smooth periodic transitions
- Activation functions influence how neural networks form approximations, but their impact is smaller than model architecture and depends on the type of function being learned

Model Approximation - Smooth x^2

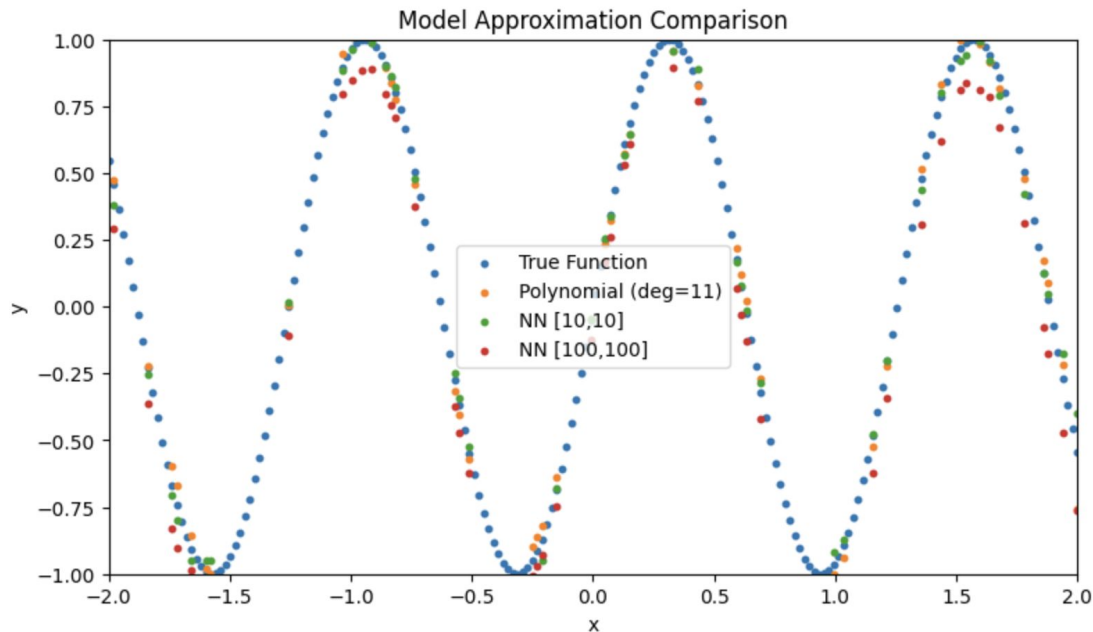


Polynomial MSE: 3.3674532216212546e-27
NN [10,10] MSE: 0.0023699130369942254
NN [100,100] MSE: 2.348666765528876e-05

Key Observations:

- Both models approximate smooth functions well
 - Polynomial regression and neural networks both capture smooth, continuous structure effectively.
- Simple structure reduces the advantage of model complexity
 - When the underlying function is simple, increasing neural network capacity does not significantly improve performance.
- Polynomial regression is especially efficient for smooth trends
 - Low-degree polynomials provide strong performance on smooth functions with minimal complexity.

Model Approximation - Oscillatory $\sin(5x)$

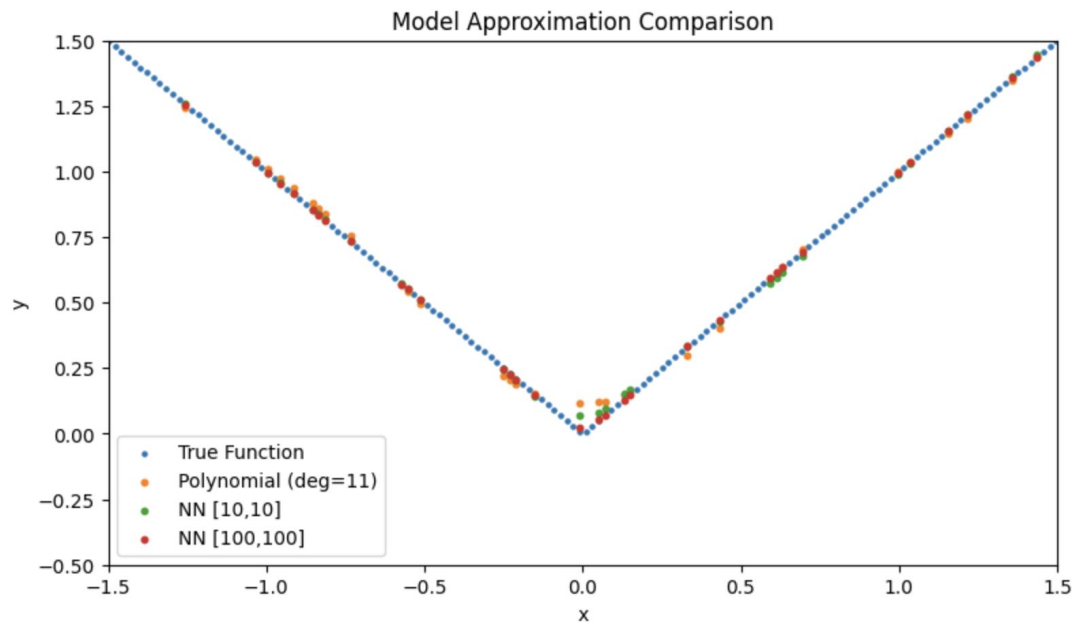


Polynomial MSE: 0.0023557286619173123
NN [10,10] MSE: 0.0022537033541523296
NN [100,100] MSE: 0.01653505037942446

Key Observations:

- Oscillatory structure is harder for polynomial regression
 - Polynomial regression struggles to consistently capture high-frequency oscillations without increasing degree, which can introduce instability.
- Neural networks adapt better to nonlinear patterns
 - Neural networks more flexibly approximate oscillatory behavior due to their layered nonlinear transformations.
- Model performance depends on frequency complexity
 - As oscillation frequency increases, models need more capacity to capture rapid changes in the function.

Model Approximation - Piecewise $|x|$



Polynomial MSE: 0.0006771096489713956
NN [10,10] MSE: 0.0001730292392737009
NN [100,100] MSE: 7.061645396217142e-06

Key Observations:

- Sharp discontinuities are difficult for smooth function approximators
 - Both polynomial regression and neural networks struggle to accurately capture the sharp corner in the piecewise function.
- Polynomial regression tends to overshoot near discontinuities
 - High-degree polynomials introduce oscillations near the kink ($|x|$), leading to instability around the corner.
- Neural networks approximate piecewise structure more flexibly
 - Neural networks generally provide a smoother approximation and better local adaptation, but may still smooth out sharp transitions.

Final Conclusions: Polynomial Regression vs Neural Networks

- No single model is best for all functions
 - Model performance depends strongly on the structure of the underlying function.
- Polynomial regression is strong for smooth, simple patterns
 - It performs efficiently on smooth functions but struggles with oscillations and discontinuities.
- Neural networks offer greater flexibility but require tuning
 - NN performance improves with capacity but shows diminishing returns and variability.
- Function complexity determines model suitability
 - Smooth → both work well
 - Oscillatory → NN advantage
 - Piecewise → both struggle at sharp transitions, but deeper neural networks have an advantage
- Overall, model selection should be guided by the structure of the data rather than model complexity alone.